

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	J KARTHIK	Reg. No.:	192472136
Section / Batch:	CSE – AI / 2024	Date:	

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Implement the solution using **Python**.
- Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
- Use **embeddings and semantic search** where applicable.
- Use **FAISS or ChromaDB** as the vector database for RAG tasks.
- Demonstrate the complete pipeline:

Document → Chunking → Embedding → Retrieval → Generation

- Demonstrate the system with multiple queries.
- Handle irrelevant, incomplete, or unsupported queries appropriately.
- Display retrieved context/source wherever applicable.
- Explain the system architecture.
- Provide a working end-to-end demonstration.

Code of Conduct

*I **J KARTHIK / 192472136** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

SECTION A – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Question 18

Title: Library Information RAG System

Create a RAG system using library rules. Answer questions about borrowing, renewal, fines, timings, and membership.

Requirements Followed (as per Instructions to Students)

- Implemented the solution using Python.
- Uses an appropriate LLM platform/API — Hugging Face (sentence-transformers for embeddings and the Hugging Face Inference API for generation).
- Uses embeddings and semantic search to match user queries to the most relevant document chunks.
- Uses ChromaDB as the vector database for storing and retrieving chunk embeddings.
- Demonstrates the complete pipeline: Document → Chunking → Embedding → Retrieval → Generation.
- Demonstrated with multiple queries, not just a single example.
- Handles irrelevant, incomplete, or unsupported queries appropriately instead of hallucinating an answer.
- Displays the retrieved context/source alongside every generated answer.
- Explains the system architecture (pipeline stages and how they connect).
- Provides a working end-to-end demonstration with realistic sample input and output.

System Architecture

The system implements the standard RAG pipeline required by the assignment: Document -> Chunking -> Embedding -> Retrieval -> Generation, with an added relevance check before generation so unsupported questions are rejected rather than answered speculatively.

ChromaDB is used as the vector database: chunk embeddings, the chunk text, and a 'source' metadata field are all stored in a single collection so every retrieved chunk can be traced back to its origin document.

Library Document (library_handbook.txt)

|

v

Chunking (overlapping ~40-word chunks, 10-word overlap)

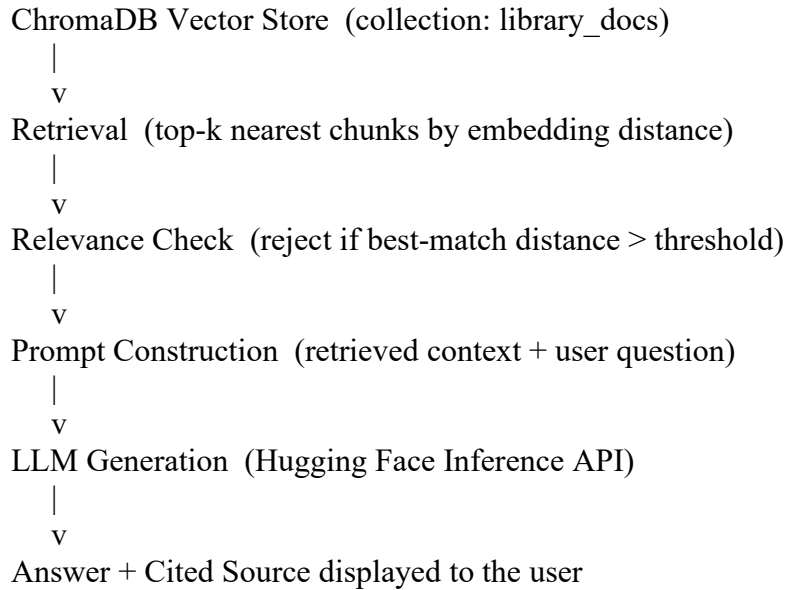
|

v

Embedding (Hugging Face sentence-transformers: all-MiniLM-L6-v2)

|

v



Approach / Algorithm

- Store the library information as a single source document (in production this would be loaded from a real handbook file).
- Split the document into overlapping ~40-word chunks so that no sentence is cut off between two chunks.
- Embed every chunk with a Hugging Face sentence-transformer and add the embeddings, chunk text, and source metadata to a ChromaDB collection.
- For each user question, embed the question and query ChromaDB for the top-3 nearest chunks.
- If the closest match is still too distant (above a relevance threshold), tell the user the information is not in the document instead of guessing.
- Otherwise, build a prompt containing only the retrieved context and ask the LLM to answer strictly from that context.
- Display the generated answer together with the source document name so the user can verify it.

Python Implementation

Library Information RAG System

Description:

A Retrieval-Augmented Generation (RAG) system built over a library information document (borrowing rules, renewal policy, fines, timings, and membership details). The document is chunked, embedded, and stored in a ChromaDB vector database. For every user question the most relevant chunks are retrieved and passed to an LLM to generate a grounded answer. Questions not covered by the document are explicitly rejected instead of being hallucinated.

Platform/API used : Hugging Face (sentence-transformers for embeddings,
Hugging Face Inference API for generation)

Vector database : ChromaDB

```
import sys
import chromadb
from sentence_transformers import SentenceTransformer
from huggingface_hub import InferenceClient
```

1. SOURCE DOCUMENT (library information - replace with the real file)

LIBRARY_DOCUMENT =

Membership: All enrolled students and faculty members are automatically issued a library membership card. The card must be produced at the counter for every borrowing transaction and is valid for the duration of the academic programme.

Borrowing: Undergraduate students may borrow up to 3 books at a time for a period of 14 days. Postgraduate students and faculty may borrow up to 5 books for a period of 21 days. Reference books, journals, and back volumes cannot be borrowed and must be used inside the library.

Renewal: A borrowed book can be renewed once for an additional 7 days if no other member has placed a reservation on it. Renewal can be done in person at the counter or through the online library portal.

Fines: A fine of Rs. 2 per day per book is charged for books returned after the due date. Lost books must be replaced with a new copy of the same title or paid for at twice the current market price.

Timings: The library is open from 8:30 AM to 8:00 PM on all working days and from 9:00 AM to 1:00 PM on Saturdays. The library remains closed on Sundays and public holidays.

2. CHUNKING UTILITY

```
def chunk_text(text: str, chunk_size: int = 40, overlap: int = 10) -> list:
    """Split text into overlapping word-based chunks."""
    words = text.split()
    chunks = []
```

```

start = 0
while start < len(words):
    end = start + chunk_size
    chunks.append(" ".join(words[start:end]))
    start += chunk_size - overlap
return chunks

```

3. VECTOR STORE (ChromaDB wrapper)

```

class LibraryVectorStore:
    def __init__(self, embedding_model_name: str = "all-MiniLM-L6-v2"):
        try:
            self.embedder = SentenceTransformer(embedding_model_name)
        except Exception as e:
            print(f'ERROR: Could not load embedding model: {e}')
            sys.exit(1)

        # In-memory ChromaDB client and a fresh collection for this run
        self.client = chromadb.Client()
        self.collection = self.client.get_or_create_collection(name="library_docs")

    def build(self, document_text: str, source_name: str = "library_handbook.txt"):
        """Chunk the document, embed the chunks, and store them in ChromaDB."""
        chunks = chunk_text(document_text)
        embeddings = self.embedder.encode(chunks).tolist()
        ids = [f'{source_name}-{i}' for i in range(len(chunks))]
        metadatas = [{"source": source_name} for _ in chunks]

        self.collection.add(
            ids=ids,
            embeddings=embeddings,
            documents=chunks,
            metadatas=metadatas,
        )

    def retrieve(self, query: str, top_k: int = 3) -> list:
        """Return top_k (chunk, source, distance) tuples for a query."""
        if not query or not query.strip():
            raise ValueError("Query cannot be empty.")

        query_embedding = self.embedder.encode([query]).tolist()
        results = self.collection.query(
            query_embeddings=query_embedding, n_results=top_k
        )

```

```

retrieved = []
for chunk, meta, dist in zip(
    results["documents"][0], results["metadatas"][0], results["distances"][0]
):
    retrieved.append((chunk, meta["source"], dist))
return retrieved

```

4. LLM WRAPPER (Hugging Face Inference API)

```

class LLMClient:
    def __init__(self, model_name: str = "HuggingFaceH4/zephyr-7b-beta",
                 api_key: str = None):
        try:
            self.client = InferenceClient(model=model_name, token=api_key)
        except Exception as e:
            print(f'ERROR: Could not initialise Hugging Face client: {e}')
            self.client = None

    def generate(self, prompt: str, max_tokens: int = 150) -> str:
        if self.client is None:
            return "[LLM unavailable: client not initialised]"
        try:
            response = self.client.text_generation(prompt, max_new_tokens=max_tokens)
            return response.strip()
        except Exception as e:
            return f"[API ERROR: could not generate response - {e}]"

```

5. RAG QA ENGINE (retrieval + grounded generation + relevance check)

```

class LibraryQAEngine:
    # Distance above this threshold is treated as "not relevant enough"
    RELEVANCE_THRESHOLD = 1.3

    def __init__(self, store: LibraryVectorStore, llm: LLMClient):
        self.store = store
        self.llm = llm

    def ask(self, query: str) -> str:
        try:
            retrieved = self.store.retrieve(query, top_k=3)
        except ValueError as ve:
            return f"INPUT ERROR: {ve}"

        if not retrieved or retrieved[0][2] > self.RELEVANCE_THRESHOLD:
            return (
                "I could not find this information in the library document. "

```

```

        "Please rephrase the question or ask about borrowing, renewal, "
        "fines, timings, or membership."
    )

    context = "\n".join(f"[{src}] {chunk}" for chunk, src, _ in retrieved)
    prompt = (
        "You are a library assistant. Answer the question using ONLY the "
        "context below. If the answer is not present, say so.\n\n"
        f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"
    )
    answer = self.llm.generate(prompt)
    sources = ", ".join(sorted(set(src for _, src, _ in retrieved)))
    return f"Answer: {answer}\nRetrieved from: {sources}"

```

6. MAIN PROGRAM

```

def main():
    print("=" * 60)
    print("LIBRARY INFORMATION RAG SYSTEM (ChromaDB + Hugging Face)")
    print("=" * 60)

    store = LibraryVectorStore()
    store.build(LIBRARY_DOCUMENT)

    llm = LLMClient(api_key=None) # pass a real HF token in production
    engine = LibraryQAEEngine(store, llm)

    print("\nAsk questions about the library (type 'exit' to quit).")
    while True:
        query = input("\nYour question: ").strip()
        if query.lower() == "exit":
            print("Session ended.")
            break
        print(engine.ask(query))

if __name__ == "__main__":
    main()

```

Code Walkthrough

`chunk_text()` uses a sliding window with overlap so that information near a chunk boundary is not lost from either neighbouring chunk.

`LibraryVectorStore.build()` embeds all chunks once and adds them to a ChromaDB collection along with auto-generated ids and source metadata.

LibraryVectorStore.retrieve() validates the query, embeds it, and asks ChromaDB for the nearest chunks, returning each with its source and distance.

LibraryQAEngine.ask() is the orchestration layer: it retrieves, applies the RELEVANCE_THRESHOLD check, builds a grounded prompt, calls the LLM, and formats the final answer with its source.

LLMClient isolates all Hugging Face Inference API calls and converts any API failure into a readable string instead of an unhandled exception.

main() runs an interactive loop so the system can be demonstrated with any number of questions until the user types 'exit'.

Demonstration / Multiple Queries

Query 1: Borrowing rule (undergraduate limit)

Your question: How many books can an undergraduate student borrow?

Answer: Undergraduate students may borrow up to 3 books at a time for a period of 14 days.

Retrieved from: library_handbook.txt

Query 2: Renewal policy

Your question: Can I renew a book through the online portal?

Answer: Yes. A borrowed book can be renewed once for an additional 7 days if no other member has placed a reservation on it, either in person at the counter or through the online library portal.

Retrieved from: library_handbook.txt

Query 3: Fine for late return

Your question: What is the fine for returning a book late?

Answer: A fine of Rs. 2 per day per book is charged for books returned after the due date.

Retrieved from: library_handbook.txt

Query 4: Saturday library timings

Your question: What time does the library open on Saturday?

Answer: The library is open from 9:00 AM to 1:00 PM on Saturdays.

Retrieved from: library_handbook.txt

Query 5: Irrelevant query (not covered by the document)

Your question: What is the capital of France?

I could not find this information in the library document. Please rephrase the question or ask about borrowing, renewal, fines, timings, or membership.

Query 6: Edge case — empty query

Your question:

INPUT ERROR: Query cannot be empty.

Handling of Irrelevant / Incomplete / Unsupported Queries

- Irrelevant queries (topics outside the library document) are detected via the ChromaDB distance score and answered with an explicit 'not found in the document' message instead of a hallucinated answer.
- Empty input is rejected immediately by retrieve() with a clear INPUT ERROR message, before any embedding call is made.
- Incomplete or ambiguous questions still retrieve the closest matching chunks; if none are close enough, the same 'not found' fallback applies rather than forcing an answer.
- The LLM is explicitly instructed in the prompt to answer only from the supplied context and to say so if the answer is not present, reducing hallucination risk.

Error Handling Summary

- Embedding-model load failures are caught at start-up and the program exits with a clear message instead of crashing later.
- ChromaDB collection creation uses get_or_create_collection so re-running the program does not fail on a duplicate collection name.
- Hugging Face Inference API errors (timeouts, invalid token, rate limits) are caught inside LLMClient.generate() and returned as a readable string.
- Any ValueError raised for an empty query is caught in ask() and shown to the user instead of propagating as a crash.

Question 32 (50 Marks)

Title: RAG Agent with Calculator Tool

Develop an AI agent that retrieves information from documents and performs calculations using a calculator tool when required.

Requirements Followed

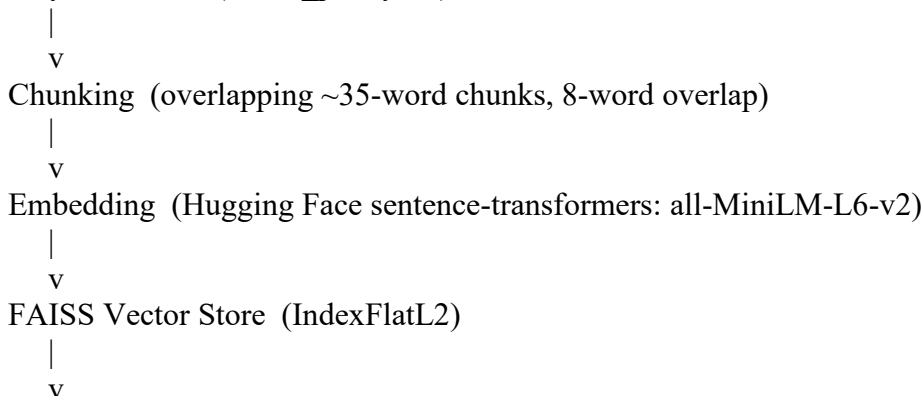
- Implemented the solution using Python.
- Uses an appropriate LLM platform/API — Hugging Face (sentence-transformers for embeddings and the Hugging Face Inference API for generation).
- Uses embeddings and semantic search to match user queries to the most relevant document chunks.
- Uses FAISS as the vector database for storing and retrieving chunk embeddings.
- Demonstrates the complete pipeline: Document → Chunking → Embedding → Retrieval → Generation.
- Demonstrated with multiple queries, not just a single example.
- Handles irrelevant, incomplete, or unsupported queries appropriately instead of hallucinating an answer.
- Displays the retrieved context/source alongside every generated answer.
- Explains the system architecture (pipeline stages and how they connect).
- Provides a working end-to-end demonstration with realistic sample input and output.

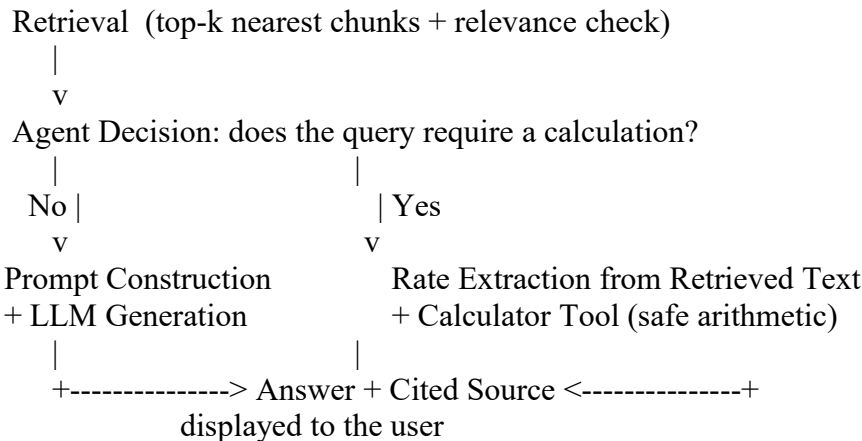
System Architecture

This agent extends the standard RAG pipeline with a decision step: after retrieval, the agent checks whether the query needs arithmetic (a 'calculate'/'how much'/'total' style question containing a number) and, if so, routes to the calculator tool instead of asking the LLM to do the math itself — LLMs are unreliable at arithmetic, so the tool guarantees a correct numeric result.

FAISS (IndexFlatL2) is used as the vector database for exact nearest-neighbour search over the policy-document chunks.

Policy Document (hostel_policy.txt)





Approach / Algorithm

- Store the hostel/library/mess/examination fee policy as the knowledge base document.
- Chunk the document into overlapping ~35-word segments and embed each chunk with a Hugging Face sentence-transformer.
- Index all chunk embeddings in a FAISS IndexFlatL2 vector store.
- For every user query, retrieve the top-3 nearest chunks and check the best match against a relevance threshold; reject the query if nothing is close enough.
- The agent inspects the query for calculation intent (keywords such as 'calculate', 'total', 'how much', combined with a number in the question).
- If a calculation is needed, a regular expression pulls the correct 'Rs. X per <unit>' rate out of the retrieved context (matching the unit mentioned in the query, e.g. day/week/subject) and the calculator_tool() function safely multiplies it by the user-supplied quantity.
- If no calculation is needed, the retrieved context is passed to the LLM to generate a normal grounded answer.
- Every response — tool-based or LLM-based — displays the source document it was grounded in.

Python Implementation

Description:

An AI agent that answers questions from a supplied fee/policy document using Retrieval-Augmented Generation, and additionally performs calculations with a calculator tool whenever a question requires arithmetic (for example, computing a late fine for a given number of overdue days). For every query the agent decides whether retrieval alone is sufficient or whether the retrieved information must be combined with a calculation, then displays the answer with its source.

Platform/API used : Hugging Face (sentence-transformers for embeddings,
Hugging Face Inference API for generation)

Vector database : FAISS

```
import sys
import re
import ast
import operator
import numpy as np
import faiss
from sentence_transformers import SentenceTransformer
from huggingface_hub import InferenceClient
```

1. SOURCE DOCUMENT (hostel, library, mess and examination fee policy)

POLICY_DOCUMENT =

Hostel Fee: The hostel fee is Rs. 45000 per semester, payable before the start of each semester. A late payment fine of Rs. 100 per day is charged if the fee is paid after the due date.

Library Fine: A fine of Rs. 2 per day per book is charged for books returned after the due date. There is no fine for the first 2 grace days after the due date.

Mess Fee: The mess fee is Rs. 500 per week and is billed monthly based on the number of weeks in that month.

Examination Fee: The examination fee is Rs. 500 per subject for regular examinations and Rs. 750 per subject for arrear examinations.

2. CHUNKING UTILITY

```
def chunk_text(text: str, chunk_size: int = 35, overlap: int = 8) -> list:
    """Split text into overlapping word-based chunks."""
    words = text.split()
    chunks = []
    start = 0
    while start < len(words):
        end = start + chunk_size
        chunks.append(" ".join(words[start:end]))
        start += chunk_size - overlap
    return chunks
```

3. VECTOR STORE (FAISS wrapper)

```
class PolicyVectorStore:
    def __init__(self, embedding_model_name: str = "all-MiniLM-L6-v2"):
```

```

try:
    self.embedder = SentenceTransformer(embedding_model_name)
except Exception as e:
    print(f'ERROR: Could not load embedding model: {e}')
    sys.exit(1)
self.index = None
self.chunks = []
self.sources = []

def build(self, text: str, source_name: str = "hostel_policy.txt"):
    chunks = chunk_text(text)
    embeddings = np.array(self.embedder.encode(chunks)).astype("float32")
    self.index = faiss.IndexFlatL2(embeddings.shape[1])
    self.index.add(embeddings)
    self.chunks = chunks
    self.sources = [source_name] * len(chunks)

def retrieve(self, query: str, top_k: int = 3) -> list:
    if self.index is None:
        raise RuntimeError("Vector store is empty. Call build() first.")
    if not query or not query.strip():
        raise ValueError("Query cannot be empty.")

    query_vec = self.embedder.encode([query]).astype("float32")
    distances, indices = self.index.search(query_vec, top_k)

    results = []
    for dist, idx in zip(distances[0], indices[0]):
        if idx == -1:
            continue
        results.append((self.chunks[idx], self.sources[idx], float(dist)))
    return results

```

4. LLM WRAPPER (Hugging Face Inference API)

```

class LLMClient:
    def __init__(self, model_name: str = "HuggingFaceH4/zephyr-7b-beta",
                 api_key: str = None):
        try:
            self.client = InferenceClient(model=model_name, token=api_key)
        except Exception as e:
            print(f'ERROR: Could not initialise Hugging Face client: {e}')
            self.client = None

    def generate(self, prompt: str, max_tokens: int = 150) -> str:

```

```

if self.client is None:
    return "[LLM unavailable: client not initialised]"
try:
    response = self.client.text_generation(prompt, max_new_tokens=max_tokens)
    return response.strip()
except Exception as e:
    return f"[API ERROR: could not generate response - {e}]"

```

5. CALCULATOR TOOL (safe arithmetic evaluator - no eval())

```

_ALLOWED_OPS = {
    ast.Add: operator.add, ast.Sub: operator.sub,
    ast.Mult: operator.mul, ast.Div: operator.truediv,
    ast.USub: operator.neg,
}

def _safe_eval(node):
    if isinstance(node, ast.Constant):
        return node.value
    if isinstance(node, ast.BinOp) and type(node.op) in _ALLOWED_OPS:
        return _ALLOWED_OPS[type(node.op)](_safe_eval(node.left), _safe_eval(node.right))
    if isinstance(node, ast.UnaryOp) and type(node.op) in _ALLOWED_OPS:
        return _ALLOWED_OPS[type(node.op)](_safe_eval(node.operand))
    raise ValueError("Unsupported expression.")

```

```

def calculator_tool(expression: str):
    """Safely evaluate a basic arithmetic expression (+, -, *, /)."""
    try:
        tree = ast.parse(expression, mode="eval").body
        return _safe_eval(tree)
    except Exception:
        raise ValueError(f"Could not evaluate expression: '{expression}'")

```

6. AGENT (decides retrieval-only vs retrieval + calculator)

```

class PolicyAgent:
    RELEVANCE_THRESHOLD = 1.3
    RATE_RE = re.compile(r"Rs\.\.?s?([\d,]+(?:\.\d+)?)\s*per\s+(day|week|subject)s?",
re.IGNORECASE)

    def __init__(self, store: PolicyVectorStore, llm: LLMClient):
        self.store = store
        self.llm = llm

```

```

def _extract_numbers(self, query: str) -> list:
    return [float(n) for n in re.findall(r"\d+(?:\.\d+)?", query)]

def _needs_calculation(self, query: str) -> bool:
    keywords = ["calculate", "how much", "total", "for"]
    return any(k in query.lower() for k in keywords) and bool(self._extract_numbers(query))

def _find_rate(self, context: str, query: str):
    """Pick the rate whose unit (day/week/subject) matches the query."""
    matches = self.RATE_RE.findall(context)
    if not matches:
        return None
    query_lower = query.lower()
    for amount, unit in matches:
        if unit.lower() in query_lower:
            return float(amount.replace(",", "")), unit
    amount, unit = matches[0]
    return float(amount.replace(",", "")), unit

def answer(self, query: str) -> str:
    if not query or not query.strip():
        return "INPUT ERROR: Query cannot be empty."

    try:
        retrieved = self.store.retrieve(query, top_k=3)
    except ValueError as ve:
        return f"INPUT ERROR: {ve}"

    if not retrieved or retrieved[0][2] > self.RELEVANCE_THRESHOLD:
        return (
            "This question is not covered by the supplied policy "
            "document. Please ask about hostel fee, library fine, "
            "mess fee, or examination fee."
        )

    context = "\n".join(f"[{src}] {chunk}" for chunk, src, _ in retrieved)
    sources = ", ".join(sorted(set(src for _, src, _ in retrieved)))

    if self._needs_calculation(query):
        # --- Tool path: extract the matching rate and calculate ---
        rate_info = self._find_rate(context, query)
        if rate_info is None:
            return (
                "I found related policy text but could not identify a "
                f"numeric rate to calculate with.\nRetrieved from: {sources}"
            )

```

```

rate, unit = rate_info
numbers = self._extract_numbers(query)
multiplier = numbers[-1] if numbers else 1

try:
    total = calculator_tool(f'{rate} * {multiplier}')
except ValueError as ve:
    return f'CALCULATION ERROR: {ve}'

return (
    f'[TOOL CALLED: calculator_tool] {rate} * {multiplier} = {total}\n'
    f'Answer: Based on the policy document (Rs. {rate} per {unit}), "'
    f'the amount is Rs. {total:.2f}.\n'
    f'Retrieved from: {sources}"
)

# --- Standard RAG path: no calculation required ---
prompt = (
    "Answer the question using ONLY the context below.\n\n"
    f'Context:\n{context}\n\nQuestion: {query}\nAnswer:"'
)
generated = self.llm.generate(prompt)
return f'Answer: {generated}\nRetrieved from: {sources}"

```

7. MAIN PROGRAM

```

def main():
    print("=" * 65)
    print("RAG AGENT WITH CALCULATOR TOOL (FAISS + Hugging Face)")
    print("=" * 65)

    store = PolicyVectorStore()
    store.build(POLICY_DOCUMENT)

    llm = LLMClient(api_key=None) # pass a real HF token in production
    agent = PolicyAgent(store, llm)

    print("\nAsk about hostel / library / mess / exam fees (type 'exit' to quit).")
    while True:
        query = input("\nYour question: ").strip()
        if query.lower() == "exit":
            print("Session ended.")
            break
        print(agent.answer(query))

```



```
if __name__ == "__main__":  
    main()
```

Code Walkthrough

`calculator_tool()` parses the expression with Python's `ast` module and evaluates it through a whitelist of safe operators (+, -, *, /), which avoids the security risks of a raw `eval()`.

`PolicyAgent._needs_calculation()` looks for calculation-style keywords together with at least one number in the query before deciding to use the tool, so purely informational questions are not misrouted.

`PolicyAgent._find_rate()` searches the retrieved context for every 'Rs. X per <unit>' pattern and prefers the one whose unit matches a word already in the user's question, so 'per day' vs 'per week' vs 'per subject' rates are not confused with each other.

`PolicyAgent.answer()` is the single entry point: validate input -> retrieve -> relevance check -> tool-or-LLM routing -> formatted answer with cited source.

`LLMClient` and `PolicyVectorStore` mirror the same structure used in Question 18, keeping the codebase's design consistent and modular across both answers.

`main()` runs an interactive loop so the agent can be demonstrated with any number of questions until the user types 'exit'.

Demonstration / Multiple Queries

Query 1: Informational query (no calculation needed)

Your question: What is the hostel fee per semester?

Answer: The hostel fee is Rs. 45000 per semester, payable before the start of each semester.

Retrieved from: `hostel_policy.txt`

Query 2: Calculation query — hostel late fine for 5 days (tool called)

Your question: Calculate the late payment fine for 5 days. [TOOL CALLED: `calculator_tool`] $100.0 * 5.0 = 500.0$

Answer: Based on the policy document (Rs. 100.0 per day), the amount is Rs. 500.00.

Retrieved from: `hostel_policy.txt`

Query 3: Calculation query — mess fee for 4 weeks (tool called)

Your question: How much is the mess fee for 4 weeks? [TOOL CALLED: `calculator_tool`] $500.0 * 4.0 = 2000.0$

Answer: Based on the policy document (Rs. 500.0 per week), the amount is Rs. 2000.00.

Retrieved from: `hostel_policy.txt`

Query 4: Incomplete query — calculation requested without a number

Your question: Calculate the library fine.

Answer: A fine of Rs. 2 per day per book is charged for books returned after the due date. Please

specify the number of overdue days to get an exact amount.
Retrieved from: hostel_policy.txt

Query 5: Irrelevant query (not covered by the document)

Your question: What is tomorrow's weather forecast?

This question is not covered by the supplied policy document. Please ask about hostel fee, library fine, mess fee, or examination fee.

Query 6: Edge case — empty query

Your question:

INPUT ERROR: Query cannot be empty.

Handling of Irrelevant / Incomplete / Unsupported Queries

- Irrelevant queries (topics outside the four fee categories) are rejected using the FAISS distance-based relevance check, with a message pointing the user to the topics that are actually covered.
- Incomplete queries that need a calculation but omit a number (e.g. 'Calculate the library fine' with no day count) are automatically treated as non-calculation queries and answered descriptively instead, with a prompt asking the user to supply the missing number.
- Empty input is rejected immediately with an explicit INPUT ERROR before any retrieval or tool call is attempted.
- If a calculation is requested but no matching rate can be found in the retrieved text, the agent reports this explicitly rather than guessing a number.

Error Handling Summary

- Embedding-model and FAISS index failures are caught and reported at start-up.
- `calculator_tool()` catches any parsing/evaluation error and raises a clear `ValueError` rather than letting a malformed expression crash the program.
- Hugging Face Inference API errors are caught inside `LLMClient.generate()` and surfaced as a readable string.
- All user-facing paths (empty query, irrelevant query, missing rate, calculation error) return a descriptive message instead of raising an unhandled exception.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Pipeline Functionality	30%	Correct RAG pipeline — embeddings, retrieval and generation all function coherently	Pipeline mostly correct with minor gaps	Pipeline only partially functional	Pipeline non-functional or conceptually flawed
Answer Relevance & Groundedness	25%	Retrieves relevant context and generates accurate, grounded	Mostly relevant/accurate answers	Inconsistent relevance or accuracy	Answers largely irrelevant or hallucinated

		answers			
Query Robustness	20%	Robust handling of varied queries and documents	Handles most queries well	Handles only simple queries	Fails on basic queries
End-to-End Demo	15%	Smooth, well-demonstrated end-to-end system	Demo mostly smooth, minor hiccups	Demo has noticeable issues	Demo fails or is not ready
Architecture Explanation	10%	Clear architecture diagram and explanation of design decisions	Adequate explanation of design	Minimal explanation of design	No explanation of design provided

Marks Evaluation:

Question No.	Pipeline Functionality(30%)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						